



Objetivo

Comprender qué son los patrones de diseño, cuándo usarlos y cuándo evitarlos.

¿Qué son los patrones de diseño?

Los **patrones de diseño** son soluciones reutilizables a problemas comunes en el diseño de software.

Historia

- **Gang of Four (GoF)** - 1994
- 23 patrones clásicos
- Enfoque en POO tradicional
- Contexto: C++, Smalltalk

Clasificación de patrones

Creacionales

Cómo crear objetos de manera flexible y reutilizable.

Estructurales

Cómo componer objetos y clases en estructuras más grandes.

Comportamentales

Cómo los objetos interactúan y distribuyen responsabilidades.

Patrones más útiles en desarrollo moderno

1. 🏭 Factory Pattern

Problema que resuelve

Crear objetos sin especificar su clase exacta.

☑ Implementación simple

```
interface Logger {
    log(message: string): void;
}

class ConsoleLogger implements Logger {
    log(message: string): void {
        console.log(`[CONSOLE] ${message}`);
    }
}

class FileLogger implements Logger {
    log(message: string): void {
        console.log(`[FILE] Writing to file: ${message}`);
    }
}

// Factory simple
class LoggerFactory {
    static create(type: 'console' | 'file'): Logger {
        switch (type) {
            case 'console':
                return new ConsoleLogger();
            case 'file':
                return new FileLogger();
            default:
                throw new Error(`Unknown logger type: ${type}`);
        }
    }
}

// Uso
const logger = LoggerFactory.create('console');
logger.log('Hello World');
```

2. 🔄 Strategy Pattern

Problema que resuelve

Cambiar algoritmos dinámicamente.

☑ Implementación práctica

```

interface PaymentResult {
  success: boolean;
  message: string;
  transactionId?: string;
}

type PaymentFunction = (amount: number) => PaymentResult;

class PaymentProcessor {
  constructor(private paymentFn: PaymentFunction) {}

  setPaymentFunction(paymentFn: PaymentFunction): void {
    this.paymentFn = paymentFn;
  }

  processPayment(amount: number): PaymentResult {
    return this.paymentFn(amount);
  }
}

// Payment function implementations
const createCreditCardPayment = (cardNumber: string): PaymentFunction => {
  return (amount: number): PaymentResult => {
    const transactionId = `cc_${Date.now()}`;
    console.log(`Paid ${amount} with credit card ending in ${cardNumber.slice(-4)}`);
    return {
      success: true,
      message: `Payment of ${amount} processed via credit card`,
      transactionId
    };
  };
};

const createPayPalPayment = (email: string): PaymentFunction => {
  return (amount: number): PaymentResult => {
    const transactionId = `pp_${Date.now()}`;
    console.log(`Paid ${amount} via PayPal (${email})`);
    return {
      success: true,
      message: `Payment of ${amount} processed via PayPal`,
      transactionId
    };
  };
};

// Usage
const processor = new PaymentProcessor(new CreditCardPayment('1234-5678-9012-3456'));
processor.processPayment(100);

processor.setStrategy(new PayPalPayment('user@example.com'));
processor.processPayment(50);

```

3. 🕒 Observer Pattern

Problema que resuelve

Notificar a múltiples objetos sobre cambios de estado.

☑ Implementación moderna

```
interface Observer<T> {
  update(data: T): void;
}

class EventEmitter<T> {
  private observers: Observer<T>[] = [];

  subscribe(observer: Observer<T>): void {
    this.observers.push(observer);
  }

  unsubscribe(observer: Observer<T>): void {
    const index = this.observers.indexOf(observer);
    if (index > -1) {
      this.observers.splice(index, 1);
    }
  }

  notify(data: T): void {
    this.observers.forEach(observer => observer.update(data));
  }
}

// Ejemplo de uso
interface UserEvent {
  type: 'created' | 'updated' | 'deleted';
  userId: string;
}

class EmailNotifier implements Observer<UserEvent> {
  update(event: UserEvent): void {
    console.log(`Sending email for user ${event.userId}: ${event.type}`);
  }
}

class AuditLogger implements Observer<UserEvent> {
  update(event: UserEvent): void {
    console.log(`Audit log: User ${event.userId} was ${event.type}`);
  }
}

// Uso
const userEvents = new EventEmitter<UserEvent>();
const emailNotifier = new EmailNotifier();
const auditLogger = new AuditLogger();
```

```
userEvents.subscribe(emailNotifier);
userEvents.subscribe(auditLogger);

userEvents.notify({ type: 'created', userId: '123' });
```

Cuándo NO usar patrones

✗ Sobre-ingeniería con patrones

```
// Singleton innecesario para configuración simple
class Config {
  private static instance: Config;
  private settings: Record<string, string> = {};

  private constructor() {}

  static getInstance(): Config {
    if (!Config.instance) {
      Config.instance = new Config();
    }
    return Config.instance;
  }

  get(key: string): string {
    return this.settings[key];
  }

  set(key: string, value: string): void {
    this.settings[key] = value;
  }
}

// Uso complicado
const config = Config.getInstance();
config.set('apiUrl', 'https://api.example.com');
```

☑ Solución simple

```
// Objeto simple para configuración
export const config = {
  apiUrl: 'https://api.example.com',
  timeout: 5000,
  retries: 3
};

// Uso directo
console.log(config.apiUrl);
```

Alternativas modernas a patrones clásicos

En lugar de Singleton

```
// ✗ Singleton clásico
class Database {
  private static instance: Database;
  // ...
}

// ✔ Módulo ES6
export const database = new Database();

// ✔ Inyección de dependencias
class UserService {
  constructor(private db: Database) {}
}
```

En lugar de Observer

```
// ✗ Observer manual
class Subject {
  private observers: Observer[] = [];
  // ...
}

// ✔ EventTarget nativo
class UserService extends EventTarget {
  createUser(user: User): void {
    // Lógica de creación
    this.dispatchEvent(new CustomEvent('userCreated', { detail: user }));
  }
}

// ✔ Reactive streams (RxJS)
import { Subject } from 'rxjs';

const userEvents$ = new Subject<UserEvent>();
userEvents$.subscribe(event => console.log('User event:', event));
```

Patrones en lenguajes modernos

JavaScript/TypeScript tiene alternativas nativas

Patrón clásico	Alternativa moderna
Iterator	<code>for...of</code> , <code>Array.map()</code>
Observer	<code>EventTarget</code> , Promises, RxJS

Patrón clásico	Alternativa moderna
Strategy	Funciones de primera clase
Template Method	Funciones de orden superior
Command	Funciones como valores

Resumen

Los patrones de diseño son **herramientas**, no **objetivos**:

- **Úsalos** cuando resuelvan problemas reales
- **Evítalos** cuando compliquen código simple
- **Prefiere** soluciones nativas del lenguaje
- **Considera** alternativas modernas (funciones, módulos, etc.)

Recuerda: El mejor patrón es el que no necesitas usar.